Министерство образования Республики Беларусь
Учреждение Образования
«Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

# Лабораторная работа № 1
по дисциплине «Жизненный цикл разработки программного обеспечения»

Выполнил
студент гр. 439401
Пуляк М.А.

Проверил
Силич С.С.

Минск 2026

**Цель работы:** разработать приложение «Калькулятор населения мира». На главной форме поместить кнопку «Ввести данные». В отдельном окне запросить у пользователя дату (не используя стандартный элемент ввода даты). Последние введенные данные нужно запоминать в главной форме и восстанавливать при повторном вводе. После этого на главной форме отобразить результат — рассчитанную численность населения в мире на указанный момент времени. Если введены некорректные данные — показать сообщение об ошибке.

## Теоретические сведения

Model-View-Controller (MVC) – это фундаментальный шаблон проектирования, предназначенный для разделения бизнес-логики приложения (компонентов работы с данными) от его визуального представления.

Согласно классической концепции, архитектура приложения разделяется на три изолированных класса:

- Модель (Model) – управляет внутренним состоянием, данными и бизнес-логикой домена приложения. Она отвечает на запросы информации о своем состоянии и обрабатывает инструкции по его изменению. Модель является полностью независимым компонентом: она ничего не знает ни о Представлении, ни о Контроллере.
- Представление (View) – отвечает за графический интерфейс (GUI), визуальное отображение данных пользователю и презентационную логику.
- Контроллер (Controller) – перехватывает действия пользователя (нажатия клавиш, клики мыши), интерпретирует их и вызывает соответствующие методы Модели для изменения её состояния.

## Активная и пассивная модели в MVC

В зависимости от способа синхронизации данных и интерфейса, автор паттерна Стив Бурбек (Steve Burbeck) выделяет две ключевые модификации MVC:

- Пассивная модель (Passive Model):

Используется в архитектурах, где Контроллер является единственным источником изменений Модели. В этом сценарии Модель полностью автономна и не способна сообщать о смене своего состояния. Контроллер сначала модифицирует Модель, а затем самостоятельно и принудительно дает команду Представлению обновить отображение на экране. Классическим примером такой схемы является стандартный протокол HTTP в веб-приложениях.

- Активная модель (Active Model):

Применяется, когда состояние Модели может меняться без прямого участия текущего Контроллера (например, при поступлении внешних данных, вычислениях по таймеру или многопоточной обработке). Так как только сама Модель способна зафиксировать изменения своего внутреннего состояния, она обязана уведомить интерфейс о необходимости синхронизации.

Чтобы сохранить независимость Модели от конкретных окон интерфейса, активная модель реализуется через паттерн Observer («Наблюдатель»), также известный как схема *«издатель-подписчик» (publish-subscribe)*. Представления регистрируются в Модели в качестве наблюдателей. При изменении бизнес-данных Модель циклически обходит список подписчиков и автоматически генерирует для них сигналы оповещения, изолируя логику расчета от визуального кода.

**Современная IDE (на примере связки VS Code и инструментария GCC/MinGW) — это комплекс программных средств для автоматизации разработки, состоящий из следующих компонентов:**

- Текстовый редактор кода — оболочка с функциями подсветки синтаксиса и структурирования исходных файлов.
- Компилятор и Линкер (g++.exe) — транслятор кода C++ в машинные объектные модули и компоновщик их с системными библиотеками в файл .exe.
- Система автоматизации сборки (tasks.json) — диспетчер, управляющий последовательностью компиляции и передачей флагов оптимизации.
- Отладчик (gdb.exe) — инструмент для пошагового выполнения программы, установки точек останова (breakpoints) и инспекции переменных в памяти.
- Менеджер расширений — подсистема интеграции плагинов (например, для поддержки UML и SysMLv2).

**Основные приемы работы с редактором кода**

Эффективное написание кода и снижение числа ошибок обеспечиваются следующими встроенными технологиями редактора:

- Автодополнение (IntelliSense) — контекстные подсказки имен функций, классов и типов данных на лету при вводе первых букв.
- Подсветка синтаксиса — цветовое выделение ключевых слов языка, типов данных, строк и комментариев для улучшения читаемости кода.
- Фоновый статический анализ (Линтинг) — мгновенное обнаружение и подчеркивание синтаксических ошибок (например, несовместимости типов в WinAPI) еще до запуска компиляции.
- Навигация и рефакторинг — быстрое перемещение к объявлению функций (клавиша F12) и безопасное массовое переименование переменных по всему проекту.
-

**Расширения для UML / SysMLv2**

В MS Visual Studio Code для работы с диаграммами UML использовано расширение «PlantUML», для работы с моделями SysMLv2 — расширение «SysML v2», предоставляющее подсветку синтаксиса и предпросмотр диаграмм системного моделирования. Для IntelliJ IDEA и Eclipse аналогами являются плагины «PlantUML integration» и «Eclipse Papyrus» соответственно.

## Код программы

```
#include <windows.h>
#include <string>
#include <cmath>
#include <stdexcept>
using namespace std;

// ========================================
// 1. МОДЕЛЬ (Model) - Бизнес-логика
// ========================================
class WorldPopulationModel {
private:
    int day = 0, month = 0, year = 0;
    long long population = 0;
    long long diffDays = 0;

    // Опорная дата: 01.01.2024
    static const int refDay = 1, refMonth = 1, refYear = 2024;
    static constexpr long long refPopulation = 8019876189LL;
    // Средний суточный прирост населения Земли (чел./сутки)
    static constexpr long long dailyGrowth = 227000LL;

    // Юлианский день (для вычисления разницы между датами)
    static long long toJulianDay(int d, int m, int y) {
        int a = (14 - m) / 12;
        int yy = y + 4800 - a;
        int mm = m + 12 * a - 3;
        return d + (153 * mm + 2) / 5 + 365LL * yy + yy / 4 - yy / 100 +
yy / 400 - 32045;
    }

public:
    HWND viewWindow = nullptr;

    void updateData(int d, int m, int y) {
        static const int daysInMonth[] =
{31,28,31,30,31,30,31,31,30,31,30,31};
        bool leap = (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
        int maxDay = daysInMonth[m - 1] + ((m == 2 && leap) ? 1 : 0);

        if (y < 1900 || y > 2100 || m < 1 || m > 12 || d < 1 || d >
maxDay) {
            throw invalid_argument("Введены некорректные данные!");
        }

        day = d; month = m; year = y;
        diffDays = toJulianDay(d, m, y) - toJulianDay(refDay, refMonth,
refYear);
        population = refPopulation + diffDays * dailyGrowth;
        if (population < 0) population = 0;

        if (viewWindow) {
            PostMessageW(viewWindow, WM_USER + 1, 0, 0);
        }
    }

    int getDay() const { return day; }
    int getMonth() const { return month; }
    int getYear() const { return year; }
    long long getPopulation() const { return population; }
};
```

```
WorldPopulationModel g_model;

// ========================================
// 2. КОНТРОЛЛЕР (Controller)
// ========================================
class WorldPopulationController {
public:
    bool trySaveData(const wstring& dStr, const wstring& mStr, const
wstring& yStr,
                     wstring& outError) {
        try {
            int d = stoi(dStr);
            int m = stoi(mStr);
            int y = stoi(yStr);
            g_model.updateData(d, m, y);
            return true;
        }
        catch (...) {
            outError = L"Ошибка ввода! Проверьте корректность даты.";
            return false;
        }
    }
};

WorldPopulationController g_controller;

HWND hDay, hMonth, hYear;
HWND hLblDate, hLblPopulation;

// Форматирование числа с разделением по разрядам (пробелами)
wstring formatNumber(long long n) {
    wstring s = to_wstring(n);
    wstring res;
    int cnt = 0;
    for (int i = (int)s.size() - 1; i >= 0; --i) {
        res = s[i] + res;
        if (++cnt % 3 == 0 && i != 0) res = L" " + res;
    }
    return res;
}

// ========================================
// 3. ПРЕДСТАВЛЕНИЕ (View) - Интерфейс
// ========================================
LRESULT CALLBACK InputWndProc(HWND hwnd, UINT msg, WPARAM wp, LPARAM lp)
{
    switch (msg) {
    case WM_CREATE: {
        CreateWindowW(L"Static", L"Дата (ДД ММ ГГГГ):", WS_VISIBLE |
WS_CHILD,
            10, 10, 150, 20, hwnd, NULL, NULL, NULL);
        hDay = CreateWindowW(L"Edit", L"", WS_VISIBLE | WS_CHILD |
WS_BORDER,
            170, 10, 30, 22, hwnd, NULL, NULL, NULL);
        hMonth = CreateWindowW(L"Edit", L"", WS_VISIBLE | WS_CHILD |
WS_BORDER,
            205, 10, 30, 22, hwnd, NULL, NULL, NULL);
        hYear = CreateWindowW(L"Edit", L"", WS_VISIBLE | WS_CHILD |
WS_BORDER,
            240, 10, 55, 22, hwnd, NULL, NULL, NULL);

        if (g_model.getYear() != 0) {
            SetWindowTextW(hDay, to_wstring(g_model.getDay()).c_str());
```

```
                SetWindowTextW(hMonth,
to_wstring(g_model.getMonth()).c_str());
                SetWindowTextW(hYear,
to_wstring(g_model.getYear()).c_str());
            }

            CreateWindowW(L"Button", L"Сохранить", WS_VISIBLE | WS_CHILD,
                105, 140, 130, 32, hwnd, (HMENU)1, NULL, NULL);
            break;
        }
        case WM_COMMAND: {
            if (LOWORD(wp) == 1) {
                wchar_t d[10], m[10], y[10];
                GetWindowTextW(hDay, d, 10);
                GetWindowTextW(hMonth, m, 10);
                GetWindowTextW(hYear, y, 10);

                wstring error;
                if (g_controller.trySaveData(d, m, y, error)) {
                    DestroyWindow(hwnd);
                } else {
                    MessageBoxW(hwnd, error.c_str(), L"Ошибка", MB_OK |
MB_ICONERROR);
                }
            }
            break;
        }
        default:
            return DefWindowProcW(hwnd, msg, wp, lp);
        }
        return 0;
    }

    LRESULT CALLBACK MainWndProc(HWND hwnd, UINT msg, WPARAM wp, LPARAM lp)
{
        switch (msg) {
        case WM_CREATE: {
            g_model.viewWindow = hwnd;
            CreateWindowW(L"Button", L"Ввести данные", WS_VISIBLE |
WS_CHILD,
                20, 20, 150, 32, hwnd, (HMENU)1, NULL, NULL);
            hLblDate = CreateWindowW(L"Static", L"Дата: -", WS_VISIBLE |
WS_CHILD,
                20, 75, 440, 20, hwnd, NULL, NULL, NULL);
            hLblPopulation = CreateWindowW(L"Static", L"Население Земли: Не
рассчитано",
                WS_VISIBLE | WS_CHILD, 20, 107, 440, 20, hwnd, NULL, NULL,
NULL);
            break;
        }
        case WM_COMMAND: {
            if (LOWORD(wp) == 1) {
                CreateWindowExW(0, L"InputWindowClass", L"Input Data",
                    WS_OVERLAPPEDWINDOW | WS_VISIBLE,
                    CW_USEDEFAULT, CW_USEDEFAULT, 340, 210, hwnd, NULL,
NULL, NULL);
            }
            break;
        }
        case WM_USER + 1: {
            wchar_t buf[16];
            swprintf(buf, 16, L"%02d.%02d.%04d", g_model.getDay(),
g_model.getMonth(), g_model.getYear());
            wstring dateText = wstring(L"Дата: ") + buf;
```

```
            wstring popText = L"Население Земли: " +
formatNumber(g_model.getPopulation()) + L" чел.";

            SetWindowTextW(hLblDate, dateText.c_str());
            SetWindowTextW(hLblPopulation, popText.c_str());
            break;
        }
        case WM_DESTROY:
            PostQuitMessage(0);
            break;
        default:
            return DefWindowProcW(hwnd, msg, wp, lp);
        }
        return 0;
    }

    int WINAPI WinMain(HINSTANCE hInst, HINSTANCE, LPSTR, int nCmdShow) {
        WNDCLASSW wcMain = {0};
        wcMain.lpfnWndProc = MainWndProc;
        wcMain.hInstance = hInst;
        wcMain.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
        wcMain.lpszClassName = L"MainWindowClass";
        RegisterClassW(&wcMain);

        WNDCLASSW wcInput = {0};
        wcInput.lpfnWndProc = InputWndProc;
        wcInput.hInstance = hInst;
        wcInput.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
        wcInput.lpszClassName = L"InputWindowClass";
        RegisterClassW(&wcInput);

        HWND hwnd = CreateWindowExW(
            0, L"MainWindowClass", L"WorldPopulationAPP",
            WS_OVERLAPPEDWINDOW | WS_VISIBLE,
            CW_USEDEFAULT, CW_USEDEFAULT, 490, 200,
            NULL, NULL, hInst, NULL
        );

        MSG msg = {0};
        while (GetMessageW(&msg, NULL, 0, 0)) {
            TranslateMessage(&msg);
            DispatchMessageW(&msg);
        }
        return 0;
    }
```

## Вывод программы

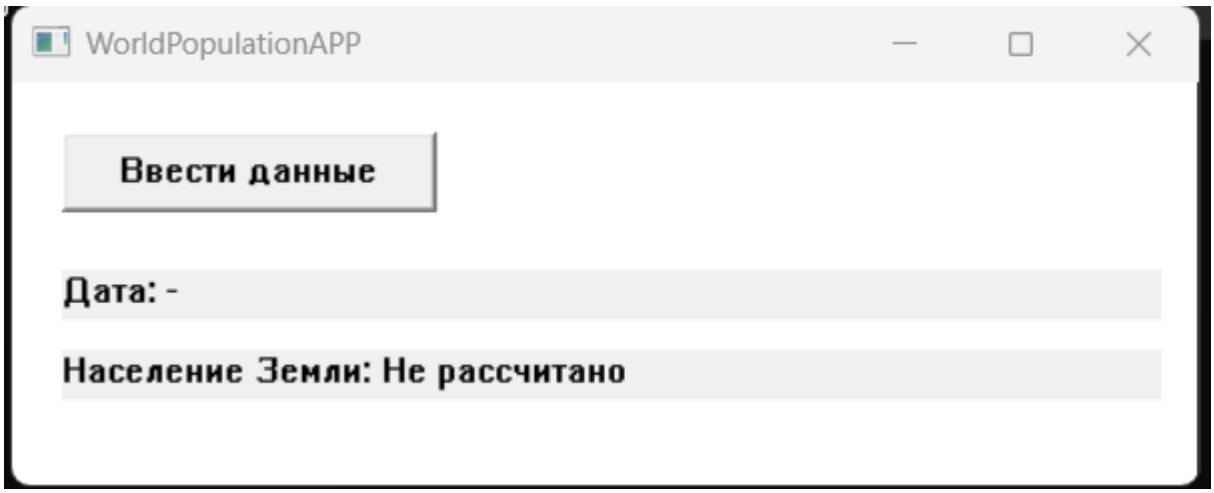



Рисунок 1 – Главный экран

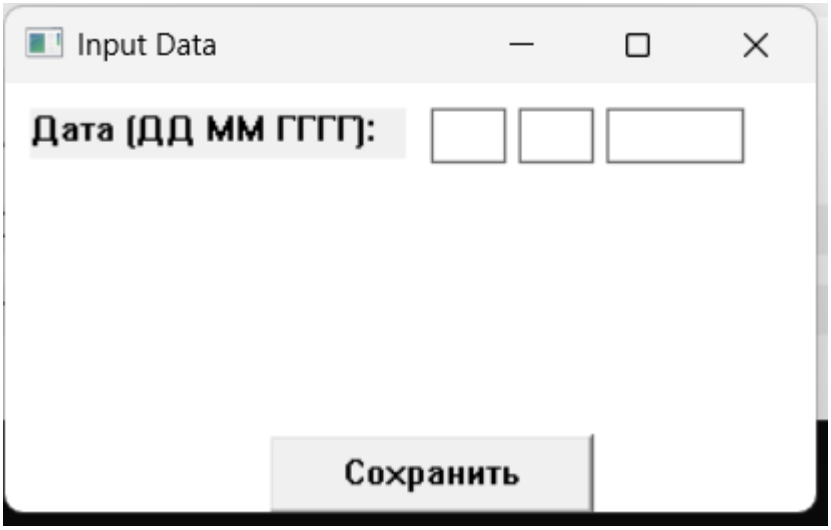



Рисунок 2 – Меню для ввода данных




Рисунок 3 – Меню после ввода данных




Рисунок 4 – Главный экран после заполнения данными

**Заключение**

В ходе выполнения лабораторной работы было разработано графическое приложение «Калькулятор населения мира». Реализована функция расчёта ориентировочной численности населения Земли на произвольную дату на основе линейной модели суточного прироста относительно опорного значения, а также механизм восстановления ранее введённых данных при повторном открытии окна ввода.

Приложение спроектировано по паттерну MVC с активной моделью: синхронизация данных и интерфейса реализована через системные сообщения Windows (PostMessageW/WM_USER) на основе паттерна «Наблюдатель», что позволяет Модели самостоятельно уведомлять Представление об изменении своего состояния. Использование низкоуровневого Windows API для построения оконного интерфейса позволило полностью выполнить требование об отказе от стандартного элемента ввода даты и от высокоуровневых фреймворков. Разработка, сборка автономного .exe файла и пошаговая отладка проекта выполняются в среде MS Visual Studio Code с компилятором MinGW (g++/gdb).